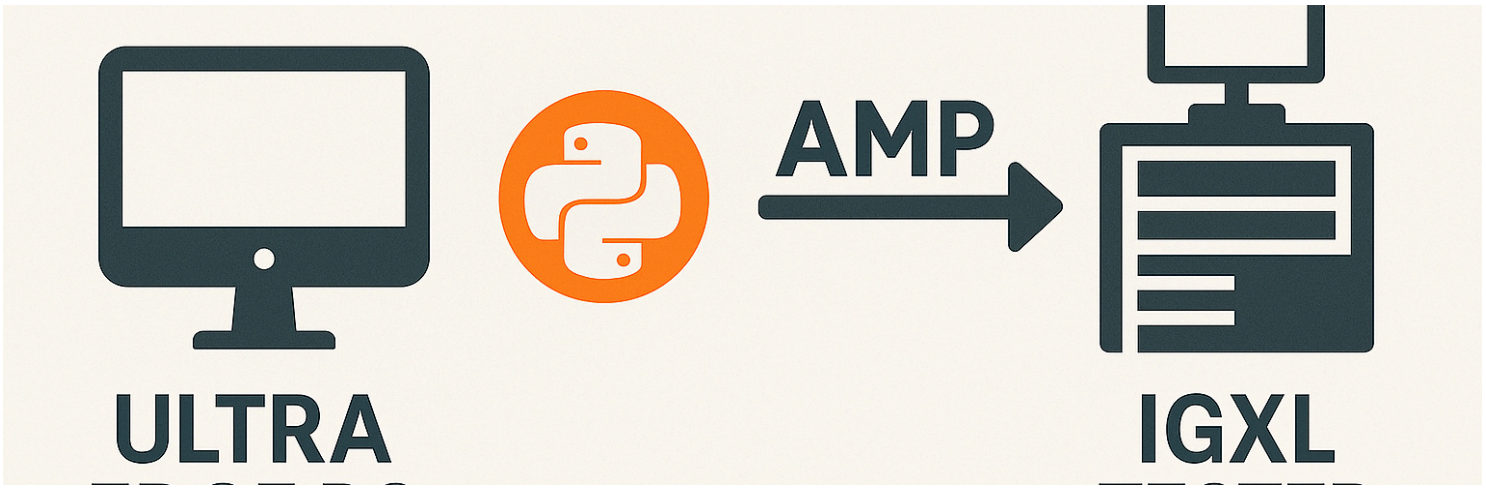

Sending AMP adaptive commands in Python

Reference : RA 465

Description

This Reference Architecture describes how to build and send AMP adaptive commands to an IGXL tester.

The code is using the Python language and is specifically adapted to an execution on the UltraEdge PC.

General Technical Info

Component	Version
Language	Python
Framework	3.11
Environment	Any
IGXL	10.60.02

Additional information about AMP adaptive testing can be found here

Source code description

Files

- AMP_adaptive_command.py
- AMP_adaptive_client.py
- sample_code.py

> **Disclaimer:** The code provided here is a sample created to clearly show what adaptive commands look like and how to send them to the RabbitMQ host on the tester. This sample code is meant to be used as a basis to develop your own code.

AMP_adaptive_command.py

(back)

This module implements classes to easily create adaptive commands in the JSON format. It supports:

- `TestControlCommand`: enable and disable tests
- `EnableWordCommand`: activate and deactive enable words
- `UpdateLimitsCommand`: update limits
- `ClearCommand`: clear previous tests enabling and update limits commands

See the API description for details.

AMP_adaptive_rmqclient.py

(back)

This module is a RabbitMQ client connecting to the RabbitMQ host located on the tester. This code uses the module `Pika` as recommended on the RabbitMQ website (see this page for example)

Parameter	**Value**
RabbitMQ Scheme	`amqp://`
Network port	5672
Username	AMP-adaptive
Password	TER-ADAPTIVE-P4SSW0RD
Queue name	AMP_DAS_Queue
Exchange name	AMP_DAS_Exchange
Exchange	
Routing key	AMP_DAS_RoutingKey
Client name	AMP Python DAS
VHost	TER-AMP-vhost

The main operations are:

- `send_action`: send an adaptive command to the tester
- `open`: setup the RabbitMQ connection
- `close`: closes the handles to the RabbitMQ connection

See the API description for details.

`sample_code.py`

(back)

This code shows how to use the `AMP_Adaptive_Command` and `AMP_Adaptive_Client` modules to create and send adaptive commands to the tester. In this sample, four commands are used:

- Enable and Disable tests
- Update limits using different parameters (based on test names or numbers, using limit names or numbers)
- Clear previous commands (note that clear does **not** apply to enable words actions)
- Enable and disable IGXL enable words

Table of Contents

AMP_adaptive_command

AdaptiveCommandsKeyNames Objects

```
class AdaptiveCommandsKeyNames()
```

List of the adaptive command key names for the JSON object sent to the RabbitMQ host

AdaptiveBaseCommand Objects

```
class AdaptiveBaseCommand()
```

Common elements to all adaptive commands

__init__

```
def __init__(DAS_URL)
```

Constructor.

Arguments:

- `DAS_URL` (`str`): identifier sent to the RabbitMQ host for traceability. It is meant to be the URL of the DAS sending the adaptive commands.

create_base

```
def create_base()
```

Base command with the fields common to all commands

EnableWordCommand Objects

```
class EnableWordCommand()
```

Adaptive command targeting enable words

__init__

```
def __init__(DAS_URL)
```

Constructor.

Arguments:

- `DAS_URL` (`str`): identifier sent to the RabbitMQ host for traceability. It is meant to be the URL of the DAS sending the adaptive commands.

activate_enable_words

```
def activate_enable_words(enablewords: list)
```

This function generates a command enabling a list of IGXL enable words.

Arguments:

- `enablewords` (`list`): list of enable words strings to activate.

deactivate_enable_words

```
def deactivate_enable_words(enablewords: list)
```

This function generates a command disabling a list of IGXL enable words.

Arguments:

- `enablewords` (`list`): list of enable words strings to deactivate.

add_action

```
def add_action(action)
```

Add an enable word action to the command

Arguments:

- `action` (`dict`): enable word action to add to the list of actions for this command.

create_command

```
def create_command()
```

Creates a JSON adaptive command string using the recorded actions

TestControlCommand Objects

```
class TestControlCommand()
```

Adaptive command to enable/disable tests

__init__

```
def __init__(DAS_URL)
```

Constructor.

Arguments:

- `DAS_URL` (`str`): identifier sent to the RabbitMQ host for traceability. It is meant to be the URL of the DAS sending the adaptive commands.

enable_testnames

```
def enable_testnames(tnames: list)
```

Enable a test using its name.

Arguments:

- `tnames` (`list`): List of test names to enable

enable_testnumbers

```
def enable_testnumbers(tnums: list)
```

Enable a test using its number.

Arguments:

- `tnums` (`list`): List of test numbers to enable

disable_testnames

```
def disable_testnames(tnames: list)
```

Disable a test using its name

Arguments:

- `tnames` (`list`): List of test names to disable

disable_testnumbers

```
def disable_testnumbers(tnums: list)
```

Disable a test using its number.

Arguments:

- `tnums` (`list`): List of test numbers to disable

add_action

```
def add_action(action)
```

Add a test enabling/disabling action to the command.

Arguments:

- `action` (`dict`): test step action to add to the list of actions for this command.

create_command

```
def create_command()
```

Creates a JSON adaptive command string using the recorded actions.

UpdateLimitsCommand Objects

```
class UpdateLimitsCommand()
```

Adaptive command to update limits for given tests

__init__

```
def __init__(DAS_URL)
```

Constructor.

Arguments:

- `DAS_URL` (`str`): identifier sent to the RabbitMQ host for traceability. It is meant to be the URL of the DAS sending the adaptive commands.

limits_command_with_names

```
def limits_command_with_names(limits: list,
                              lo_lim: float,
```

```
                                hi_lim: float,
                                llm_scale: int = None,
                                hlm_scale: int = None)
```

Creates a portion of a limit command focused on a specific limit update.

Arguments:

- `limits` (`list`): list of names as found in the "Use-Limit" IGXL flow entries.
- `lo_lim` (`float`): new low limit
- `hi_lim` (`float`): new high limit
- `llm_scale` (`int`): scaling value for the low limit (0 = no scaling)
- `hlm_scale` (`int`): scaling value for the high limit (0 = no scaling)

limits_command_with_numbers

```
def limits_command_with_numbers(limits: list,
                                lo_lim: float,
                                hi_lim: float,
                                llm_scale: int = None,
                                hlm_scale: int = None)
```

Creates a portion of a limit command focused on a specific limit update.

Arguments:

- `limits` (`list`): list of numbers as found in the "Use-Limit" IGXL flow entries.
- `lo_lim` (`float`): new low limit
- `hi_lim` (`float`): new high limit
- `llm_scale` (`int`): scaling value for the low limit (0 = no scaling)
- `hlm_scale` (`int`): scaling value for the high limit (0 = no scaling)

update_limits_with_testnames

```
def update_limits_with_testnames(tnames: list, limits_cmd: dict)
```

Update limits for a list of test names.

Arguments:

- `tnames` (`list`): list of test names on which to update the limits.
- `limits_cmd` (`dict`): update limits command to apply the list of test names.

update_limits_with_testnumbers

```
def update_limits_with_testnumbers(tnums: list, limits_cmd: dict)
```

Update limits for a list of test numbers.

Arguments:

- `tnums` (`list`): list of test number on which to update the limits.
- `limits_cmd` (`dict`): update limits command to apply the list of test number.

add_action

```
def add_action(action)
```

Add an enable word action to the command.

Arguments:

- `action` (`dict`): single update limits action to perform.

create_command

```
def create_command()
```

Creates a JSON adaptive command string using the recorded actions

ClearCommand Objects

```
class ClearCommand()
```

__init__

```
def __init__(DAS_URL)
```

Constructor.

Arguments:

- `DAS_URL` (`str`): identifier sent to the RabbitMQ host for traceability. It is meant to be the URL of the DAS sending the adaptive commands.

create_command

```
def create_command()
```

Creates the JSON adaptive command string to clear previous commands.

Table of Contents

AMP_adaptive_rmqclient

RabbitMQAdaptiveActionSender Objects

```
class RabbitMQAdaptiveActionSender()
```

RabbitMQ producer code to send adaptive commands in the form of JSON strings

__init__

```
def __init__(amp_server_address)
```

Constructor

Arguments:

- `amp_server_address` (`str`): RabbitMQ host IP address.

is_open

```
@property
def is_open()
```

Is the connection open or not?

send_action

```
def send_action(message)
```

Main feature of this class.

Arguments:

- `message` (`str`): JSON adaptive command as created by one of the classes in the AMP_adaptive_command module.

open

```
def open()
```

Initializes the communication

close

```
def close()
```

Closes all RabbitMQ communication objects

Description	Download Link
`RA_0465.zip`	Download
GIT Hub Link	Git Hub

More details about those source code here

This documentation is part of the Teradyne Archimedes Reference Architecture series.